

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА



Отчёт по лабораторной работе № 1
«Вёрстка сайта средствами HTML, CSS и Bootstrap»
по дисциплине «Web-разработка: Frontend»
Семестр 4

Выполнил: студент
Мавров Артём Николаевич
гр. J3213
ИСУ 466574

Отчёт сдан: 16.03.2026

Проверил: Добряков Д. И.

Санкт-Петербург
2026

Содержание

1	Задача	3
2	Ход работы	3
2.1	Структура проекта	3
2.2	Вёрстка и стилизация	3
2.3	Экраны аутентификации	4
2.4	Навигация между разделами	5
2.5	Раздел «Эксперименты» с фильтрацией	5
2.6	Страница эксперимента	5
2.7	Model Registry	6
2.8	Артефакт-стор	6
2.9	Личный кабинет	7
3	Вывод	7

Задача

В рамках данной лабораторной работы выбран вариант:

Платформа для управления ML-пайплайнами (аналог MLflow).

Необходимо выполнить вёрстку сайта средствами HTML, CSS и Bootstrap, продумать и реализовать интерактивные элементы на JavaScript.

Обязательные разделы платформы:

1. Вход в систему.
2. Регистрация нового пользователя.
3. Личный кабинет пользователя (эксперименты, модели, артефакты).
4. Поиск экспериментов с фильтрацией по дате, метрикам и тегам.
5. Страница эксперимента (логи, метрики, артефакты, визуализации).
6. Управление моделями (версионирование, развёртывание, артефакт-стор).

Ход работы

Структура проекта

Проект реализован в виде одностраничного приложения (SPA) на трёх файлах:

- `index.html` — разметка всех экранов приложения;
- `style.css` — стили;
- `app.js` — вся логика навигации, авторизации и работы с данными.

Вёрстка и стилизация

Для стилизации использованы Bootstrap (сетка, модальные окна, утилиты) и собственный файл `style.css`. Выбрана тёмная тема: тёмный фон, синий акцент, моноширинные шрифты для метрик и идентификаторов.

В CSS определены переменные для всей цветовой схемы:

```
1 :root {
2   --bg:    #0f1117;    /*
3   --bg2:   #1a1d27;    /*
4   --bg3:   #22263a;    /*
5   --brd:   #2a2f45;    /*
6   --acc:   #4f8ef7;    /*
7   --ok:    #3ecf8e;    /*
8   --err:   #f06080;    /*
9 }
```

Листинг 1: Переменные цветовой схемы в `style.css`

Лейаут приложения построен на фиксированном боковом меню шириной 200 px и основной области, сдвинутой на то же расстояние:

```

1 .app-shell { display: flex; min-height: 100vh; }
2 .sidebar   { width: 200px; position: fixed;
3             background: var(--bg2); }
4 .main-wrap { margin-left: 200px; flex: 1; }

```

Листинг 2: Базовый лейаут app-shell

Экран аутентификации полностью изолирован от app-shell. Центрирование формы входа реализовано через flexbox:

```

1 .auth-screen {
2   min-height: 100vh;
3   display: flex;
4   align-items: center;
5   justify-content: center;
6 }
7 .auth-center { width: 100%; max-width: 400px; }

```

Листинг 3: Центрирование формы входа

Экраны аутентификации

На странице входа пользователь вводит email и пароль. Валидация выполняется в JavaScript: если пользователь не найден в массиве `USERS`, выводится сообщение об ошибке. Для быстрого тестирования добавлена кнопка «Войти как demo», которая автоматически заполняет поля и выполняет вход:

```

1 function fillDemo() {
2   document.getElementById('li-email').value
3     = 'admin' + '@' + 'ml.pipe';
4   document.getElementById('li-pass').value = 'admin123';
5   doLogin();
6 }

```

Листинг 4: Функция быстрого входа под demo-аккаунтом

Адрес email намеренно собирается конкатенацией строк, чтобы избежать автоматической обфускации со стороны CDN-прокси Cloudflare.

Функция `doLogin()` ищет совпадение в массиве пользователей и при успехе вызывает `startApp()`, которая переключает видимость экранов:

```

1 function startApp() {
2   document.getElementById('screen-auth')
3     .style.display = 'none';
4   var app = document.getElementById('screen-app');
5   app.style.display = 'flex';
6   app.classList.remove('d-none');
7   document.getElementById('sb-name').textContent
8     = currentUser.name;
9   nav('dashboard', document.querySelector('.nav-item'));
10 }

```

Листинг 5: Переключение экранов при входе

На странице регистрации реализована проверка заполненности полей, минимальной длины пароля (6 символов) и уникальности email. При успехе новый пользователь добавляется в массив и происходит автоматический вход.

Навигация между разделами

Все разделы приложения — блоки `<div class="view">`. Переход между ними реализован скрытием всех блоков и показом нужного:

```
1 function nav(view, el) {
2   document.querySelectorAll('.view').forEach(function(v) {
3     v.classList.add('d-none');
4   });
5   document.getElementById('view-' + view)
6     .classList.remove('d-none');
7   document.querySelectorAll('.nav-item').forEach(
8     function(n) { n.classList.remove('active'); }
9   );
10  if (el) el.classList.add('active');
11 }
```

Листинг 6: Функция навигации nav()

Раздел «Эксперименты» с фильтрацией

Список экспериментов хранится в массиве объектов. Каждый эксперимент содержит имя, дату, теги, количество запусков и лучшую метрику. Реализована одновременная фильтрация по нескольким критериям: имени, дате (от/до), тегу и значению метрики.

```
1 var list = experiments.filter(function(e) {
2   if (name && e.name.toLowerCase()
3       .indexOf(name) === -1) return false;
4   if (from && e.date < from) return false;
5   if (to && e.date > to) return false;
6   if (tag && e.tags.toLowerCase()
7       .indexOf(tag) === -1) return false;
8   if (metric) {
9     var m = metric.match(/([a-z]+)([>=])([\d.]+)/);
10    if (m) {
11      var re = new RegExp(m[1] + '=(' + m[2] + '[\d.]+)');
12      var res = e.best.match(re);
13      if (res) {
14        var val = parseFloat(res[1]);
15        if (m[2] === '>' && !(val > +m[3])) return false;
16        if (m[2] === '>=' && !(val >= +m[3])) return false;
17        if (m[2] === '<' && !(val < +m[3])) return false;
18      }
19    }
20  }
21  return true;
22 });
```

Листинг 7: Фрагмент функции filterExps()

Фильтр метрик поддерживает операторы сравнения: `acc>0.9`, `f1>=0.8`, `loss<0.3`.

Страница эксперимента

При клике на эксперимент открывается детальная страница с четырьмя вкладками: **Логи**, **Метрики**, **Артефакты**, **Визуализации**. Переключение вкладок реализовано

без сторонних библиотек:

```
1 function expTab(el, tab) {
2   document.querySelectorAll('.tab-item').forEach(
3     function(t) { t.classList.remove('active'); }
4   );
5   document.querySelectorAll('.tab-panel').forEach(
6     function(p) { p.classList.add('d-none'); }
7   );
8   if (el) el.classList.add('active');
9   document.getElementById('tab-' + tab)
10     .classList.remove('d-none');
11 }
```

Листинг 8: Переключение вкладок страницы эксперимента

- **Логи** — консольный вывод обучения с цветовой подсветкой уровней (INFO, WARN, OK).
- **Метрики** — карточки метрик и bar-chart потерь на чистом CSS.
- **Артефакты** — файлы, отфильтрованные по id эксперимента из глобального массива.
- **Визуализации** — заглушки ROC-кривой и матрицы ошибок.

Model Registry

Раздел управления моделями реализует жизненный цикл: *Development* → *Staging* → *Production* → *Archived*. Функция Promote переводит модель на следующий этап:

```
1 function promoteModel(i) {
2   var order = ['dev', 'staging', 'prod'];
3   var idx = order.indexOf(models[i].stage);
4   if (idx < order.length - 1) {
5     models[i].stage = order[idx + 1];
6     renderModels();
7   }
8 }
```

Листинг 9: Продвижение модели по стадиям

Развёртывание модели открывает Bootstrap-модальное окно, в котором выбирается целевая среда (Docker, Kubernetes, FastAPI) и задаётся количество реплик.

Артефакт-стор

Отдельный раздел отображает все артефакты со всех экспериментов. Реализована фильтрация по имени файла и типу (model, dataset, plot, config). Загрузка нового артефакта осуществляется через файловый диалог:

```
1 <div class="upload-zone"
2   onclick="this.querySelector('input').click()">
3   <span>
4     </span>
5     <input type="file" class="d-none" id="ua-file">
```

```
5         onchange="previewFile(this)">
6     </div>
```

Листинг 10: Зона загрузки файла в артефакт-стор

Личный кабинет

Раздел «Мой кабинет» показывает данные, отфильтрованные по полю `owner === currentUser.name`: эксперименты, модели и артефакты текущего пользователя. Здесь же доступен API-токен с возможностью показать и скопировать через `navigator.clipboard.writeText`.

Вывод

В ходе лабораторной работы разработан интерфейс платформы управления ML-пайплайнами, выполняющий функции аналога MLflow. Реализованы все шесть обязательных разделов: аутентификация, регистрация, личный кабинет, каталог экспериментов с фильтрацией, детальная страница эксперимента и Model Registry с артефакт-стором.

Вёрстка выполнена средствами HTML5, CSS3 и Bootstrap 5. Интерактивность реализована на чистом JavaScript без сторонних библиотек: переключение экранов SPA, многопараметрическая фильтрация с операторами сравнения, жизненный цикл моделей, модальные окна Bootstrap для создания сущностей и развёртывания моделей.